Parte 8. Banco de preguntas y puntos débiles

Guía de estudio del sistema Triverde · Grupo 26 · Ingeniería de Software

Contenido

- 1. Respuestas de un minuto
- 2. Banco de preguntas transversales
- 3. Puntos débiles conocidos y cómo responder
- 4. Si piden ver el sistema
- 5. Si piden ver un archivo
- 6. Cifras para memorizar

Esta parte reúne las preguntas transversales que pueden aparecer en la presentación final, los puntos débiles conocidos con la forma correcta de responderlos, y qué abrir si se pide ver el sistema o un archivo. Las preguntas específicas de cada tema están al final de las Partes 1 a 7.

1. Respuestas de un minuto

¿Qué es el proyecto? Un sistema de gestión operativa para Triverde, una planta de procesamiento de material orgánico en Colina. Registra la recepción de camiones, el inventario y las pilas de compostaje, la parte comercial, la trazabilidad para SINADER, el mantenimiento de la maquinaria y los documentos legales. 12 módulos, 92 casos de uso, 59 requerimientos funcionales, cuatro incrementos.

¿Qué problema resuelve? La falta de un registro operativo confiable y trazable. Sin él la empresa no tiene historial financiero ni trazabilidad, perdió un fondo de 30 millones y no pudo aceptar a un municipio como cliente.

¿Cómo está construido? PWA en React que funciona sin señal, API REST en Django REST Framework y base relacional PostgreSQL, todo en Docker Compose. Tres capas: Vistas, Controladores y Persistencia.

¿Qué es lo más valioso técnicamente? La operación sin conexión con sincronización idempotente (no duplica aunque se reenvíe) y la trazabilidad del lote vendido hasta las descargas de origen.

¿Cómo trabajaron? Scrum++ con cuatro sprints, un incremento funcional por sprint aceptado por el Product Owner, ramas y pull requests por módulo, integración con la suite completa en verde.

2. Banco de preguntas transversales

Sobre el cliente y el análisis

¿Cómo levantaron los requerimientos? Entrevistas y reuniones con Javier, audios y documentos que él entregó (Excel de densidades, croquis, tarifas). Se ordenaron en UR, RF y CU, y Javier aprobó el levantamiento por correo.

¿Cómo validaron con el cliente? Javier aceptó cada incremento en el Sprint Review, y en el Inc 4 graba un video usando el sistema.

¿Cuál es el caso de uso más importante? CU-25/CU-29 (recepción con peso derivado), porque ahí nace todo el dato. Para el negocio B2B, CU-68 (trazabilidad del lote), porque es lo que exige la Ley REP.

¿Por qué el sistema tiene 12 módulos si hoy solo tritura ramas? Porque se diseñó para la planta de compostaje que está tramitando: así, cuando llegue el permiso, no hay que rediseñar nada.

¿Qué pasa si el factor de CO_2 nunca llega? El indicador muestra los cálculos pendientes con su motivo. No se inventan valores; al configurarlo se calcula todo de forma automática.

Sobre requerimientos y diseño

¿Qué diferencia hay entre caso de uso y requerimiento funcional? El RF dice qué funcionalidad debe tener el sistema; el CU describe la interacción concreta actor-sistema que lo realiza, con su flujo y sus excepciones.

¿Qué es la vista 4+1? Cinco vistas de la arquitectura: escenarios (casos de uso), lógica (modelo y clases), procesos (secuencias), desarrollo (componentes) y física (despliegue).

¿Por qué los CU no mencionan la base de datos? Porque deben ser independientes de la implementación. Hablan de objetos del dominio.

¿Qué es 3FN? Tercera forma normal: atributos atómicos, sin dependencias parciales de la clave y sin dependencias entre atributos que no son clave.

¿Qué es REST? Un estilo de API donde cada recurso tiene una dirección (`/api/v1/recepciones/`) y se opera con los métodos HTTP: GET lee, POST crea, PUT o PATCH modifica, DELETE elimina.

Sobre la construcción

¿Qué pasa si dos operadores registran al mismo tiempo sobre el mismo inventario? El servicio de inventario abre una transacción y bloquea la fila del saldo (`select_for_update`), así que el segundo espera al primero. Nunca quedan saldos negativos.

¿Qué pasa si se corta la luz a mitad de una operación? Las operaciones van en transacción: o se guarda todo o nada. Si se cortó la señal, la operación quedó en la cola del dispositivo y se envía después.

¿Cómo se replicaría a otra planta? Todo valor de negocio (densidades, tarifas, recetas, factores, umbrales) es configurable. Se instala con Docker Compose y se cargan los parámetros de la otra planta.

¿Dónde se guardan las fotos? Llegan en base64, se validan como imagen real y se guardan como archivo en el almacenamiento de medios del servidor; la tabla guarda la referencia.

¿Cómo se protege la contraseña? Django la guarda con hash (PBKDF2); nunca se guarda ni viaja en texto después del login. Las peticiones siguientes usan el token.

¿Qué pasa si un usuario escribe a mano la dirección de una pantalla que no le corresponde? La pantalla puede cargar, pero la API responde 403. La seguridad está en el servidor.

¿Qué harían distinto? Fijar desde el primer Planning los contratos de objetos compartidos (la colisión de la tabla de alertas en el Inc 2 se habría evitado) y poner un punto de control a mitad de cada sprint.

Sobre pruebas y calidad

¿Cómo saben que funciona? 263 pruebas del servidor contra PostgreSQL real, 38 escenarios de extremo a extremo con navegador, 92,4 % de cobertura y regresión completa en cada incremento.

¿Qué norma de calidad usaron? ISO/IEC 25010:2023, revisando cada característica con evidencia.

¿Cómo demuestran que la lógica no está en la pantalla? Con las trazas por CU que deja el servidor y que leen las pruebas de extremo a extremo.

Sobre la metodología

¿Cómo se repartió el trabajo? Por módulos en el Inc 2 y por parejas en el Inc 3, con especificaciones que fijaban el contrato de datos. El jefe de proyecto revisó e integró cada entrega.

¿Cómo se demuestra que todos participaron? Con el informe de commits de cada incremento y el Excel de organización del trabajo (quién hizo y quién revisó cada parte).

3. Puntos débiles conocidos y cómo responder

Responder siempre con los hechos, sin culpar a nadie y sin inventar.

#	Punto débil	Cómo responder
1	**Algunos CU funcionan en el servidor pero no tienen pantalla propia en las Vistas:** recetas, costo por km, costos operativos e historial de parámetros (CU-10, 12, 13, 14); inspección con aceptación, rechazo y multa (CU-30, 31); creación de pilas, composición, procesos y trazabilidad interna (CU-35 a 43)	La regla de negocio, la API y las pruebas de esos CU están implementadas y aprobadas (se pueden mostrar las pruebas y la API). Lo pendiente es su pantalla, que se trabaja en el cierre
2	**Factor de CO_2 pendiente del cliente**	Es la excepción del CU, no una falla: el sistema no inventa el factor y muestra los pendientes
3	**Criterio de asignación de descargas a pilas** (qué descarga alimenta cada pila) por validar con el cliente	El sistema registra el aporte que indica el operador; el criterio de negocio lo confirma el cliente
4	**HH efectivas estimadas al cierre**, no registradas a diario	Se estimaron con la evidencia del repositorio (PR, fechas, retrabajo)
5	**El sistema no está en un servidor de producción**	Está empaquetado en Docker Compose y se instala igual en cualquier servidor. La puesta en producción se planificó para cuando la planta opere con su resolución sanitaria; hoy el cliente está en esa tramitación

#	Punto débil	Cómo responder
6	**Compatibilidad no probada en Safari de iPhone**	Las pruebas de extremo a extremo corren en Chromium; queda como observación en la evaluación ISO
7	**Inventario y recepciones modificadas después:** si una recepción se rechaza después de crear sus líneas, o se edita una línea ya contabilizada, el inventario no se revierte solo	Es una limitación documentada en la Decisión 6 del modelo de datos
8	**Cobertura más baja en algunos archivos** (vistas de mantenimiento y mantenedores, servicio de proyecciones)	Están sobre 70 %; el total es 92,4 %
9	**Entregas tardías o incompletas en el Inc 3**	Se completaron en la rama de cada autor conservando sus commits; está en la retrospectiva con sus acciones
10	**Nombres de controladores que difirieron del catálogo** en algunas especificaciones	Se alinearon al catálogo en el código y en las secuencias antes de la entrega del Inc 3

4. Si piden ver el sistema

Paso	Cómo
Levantar	Desde `backend/`: `docker compose up -d`. Desde `frontend/`: `npm run dev -- --host`
Datos de demostración	`docker compose exec -T web python manage.py seed_demo` (deja usuarios y datos de todos los módulos)
Recorrido corto	Panel de control → registrar una recepción y ver el peso calculado → inventario → mezcla y alertas → cotizador → venta y cobro → certificados → trazabilidad del lote → reportes y exportación → documentos y cumplimiento
Si algo falla en vivo	Mostrar la misma funcionalidad desde su prueba o desde la API (`/api/v1/` lista todos los recursos)

Conviene levantar todo antes de que empiece la presentación.

5. Si piden ver un archivo

Mostrar siempre desde el **repositorio del grupo**. Atajo: buscar en todo el proyecto `CU-NN` y abrir el servicio o la prueba que aparezca.

Si piden…	Abrir
Un modelo	`backend/<app>/models.py`

Si piden…	Abrir
Un controlador	`backend/<app>/views.py`
La regla de un CU	`backend/<app>/services.py` (y `certificados.py`, `sinader.py`, `cumplimiento.py` según el módulo)
La conversión a peso	`backend/recepcion/serializers.py`, `derivar_peso_y_chip`
La sincronización	`backend/recepcion/views.py`, `SincronizacionView`; `frontend/src/lib/sync.ts`
El inventario	`backend/inventario/services.py`, `aplicar_movimiento`
Las rutas de la API	`backend/config/urls.py` y el `urls.py` de cada app
Permisos, auditoría, trazas	`backend/common/permissions.py`, `auditoria.py`, `trazas.py`
Una migración	`backend/<app>/migrations/`
Las pruebas	`backend/<app>/tests.py`; extremo a extremo en `frontend/e2e/`
Una pantalla	`frontend/src/pages/...`; rutas en `frontend/src/App.tsx`
Una secuencia	`docs/trabajo-grupal/diagramas/secuencias/`
Un CU extendido	`docs/referencia/Casos de uso extendidos.pdf`
Quién hizo qué	Pull requests y commits en GitHub; informe de commits de cada incremento

Si no se recuerda algo en el momento, es mejor abrir el archivo y leer el comentario inicial de la función (todos explican su CU y sus excepciones) que adivinar.

6. Cifras para memorizar

Cifra	Valor
Módulos · UR · RF · RNF · CU	12 · 28 · 59 · 12 · 92
CU por incremento	29 · 43 · 20 · 0 (acumulado 29 · 72 · 92)
Tablas	41
Controladores de diseño · vistas	26 · unas 55
Diagramas de secuencia	184
Pruebas del servidor · extremo a extremo · cobertura	263 · 38 · 92,4 %
HH del proyecto · valor	840 · $15.000 → $12.600.000
HH estimadas / efectivas a Inc 3	770 / 825
Operación anual	$133.000
Beneficio operativo anual · B/C · recuperación	$3.070.000 · 23:1 · 4,1 años

Cifra	Valor
Fondo perdido	$30.000.000
Tarifas	10 m³ $20.000 · 20 m³ $30.000 · 30 m³ $40.000 · 40 m³ $50.000
Mezcla · rendimiento · sacos/m³ · techo de temperatura	1:3 · 0,6 · 25 · 70 °C
Etapa 1	300 t/mes → 180 t → 360 m³ → 9.000 sacos → ~$27 millones/mes